

ClickSafe Project Details

Project Snapshot

ClickSafe is an AI-powered phishing-risk console. A user submits a URL before visiting it; ClickSafe validates the destination, visits it in an automated browser, gathers technical and reputation evidence, and returns a **Safe**, **Suspicious**, or **Malicious** verdict with a risk score from 0 to 100.

Implementation status: **Phases 1 through 8 are complete.** The committed roadmap is complete. Two later checks now run in front of that pipeline and are stored on the same job: a local URL pre-scan and a local phishing classifier. Both are evidence. Neither one sets the verdict.

ClickSafe is an evidence-assisted security tool. Its verdict is a risk assessment, not a guarantee that a website is harmless.

Analysis payloads still store `lifecycle.phase: 8`. The newer checks were added inside that stage. They did not add a public endpoint, a verdict label, or a database table. A beginner-friendly snapshot is in [PROGRESS.md](#).

Product Goals

- Help users make a safer decision before opening an unfamiliar link.
- Combine URL-text, classifier, browser, technical, reputation, and AI evidence rather than relying on one signal.
- Preserve enough evidence for a user or security analyst to understand the verdict.
- Keep the system modular so analyzers and data providers can be changed independently.
- Provide a responsive dashboard that works in light and dark modes.

Current User Flow

1. A user enters an HTTP/HTTPS URL or a bare domain in the React dashboard.
2. The frontend sends the URL to `POST /api/v1/analyze`.
3. The backend creates an analysis job in SQLite, then runs the local URL pre-scan and the local phishing classifier. A failure in either check is saved and does not stop the scan.
4. The backend validates and normalizes the URL, then rejects local, private, and other restricted destinations.
5. Playwright loads the destination in a new Chromium browser context.
6. ClickSafe records the redirect chain, final URL, response status, page HTML, and a full-page screenshot.
7. Technical analyzers and reputation providers collect evidence.
8. The evidence bundle, including the pre-scan and classifier results, is assessed with the OpenAI Responses API when an API key is configured. A clearly labeled local heuristic fallback is used when it is not.
9. The job, verdict, score, explanation, and evidence are persisted and shown in the dashboard.

Implemented Features

URL Validation and Job Lifecycle

- Accepts HTTP and HTTPS URLs, including bare domains such as `example.com`.
- Rejects unsupported schemes such as `ftp`.
- Enforces a request URL length between 1 and 2,048 characters.
- Normalizes host casing, removes fragments, removes default ports, and adds HTTPS to a bare domain.
- Persists the job lifecycle as `requested`, `running`, `completed`, or `failed`.
- Returns useful persisted failure information for validation and browser-navigation errors.
- Keeps the pre-scan and classifier results on the job when validation or browser navigation fails.

URL Pre-Scan

`UrlPreScanner` in `backend/src/clicksafe/application/services/url_pre_scanner.py` reads the submitted URL as text. It does not fetch the page, and it does not produce the ClickSafe verdict. The dashboard shows this block as **Initial URL risk**.

The result stores `risk_score`, `risk_level`, `features`, `suspicious_keywords`, `brand_matches`, and `warnings`.

Score	Level
0–24	low
25–54	medium
55–100	high

Signal	Points
Hostname is an IP address	30
URL contains <code>@</code>	30
A known brand token is not on that brand's official domain	25
Hostname uses Punycode (<code>xn--</code>)	20
Top-level domain is on the suspicious list	15
Domain is a known URL shortener	15
URL contains percent-encoded characters	10
Hostname has 4 or more subdomains	10
URL length is 150 characters or more	8
Domain length is 50 characters or more	8
URL contains 7 or more dots, 5 or more hyphens, 15 or more digits, or 20 or more special characters	6
Words such as <code>login</code> , <code>account</code> , <code>secure</code> , or <code>payment</code>	0

Keyword matches are saved and named in a warning. Their score contribution is 0, because a legitimate site can contain those words. If the scanner throws, the job stores a low score of 0 and a warning, and the rest of the scan continues.

Local Phishing Classifier

`MLPhishingService` calls the saved model in `backend/src/clicksafe/infrastructure/ml/models/model.joblib`. The pipeline is TF-IDF plus logistic regression, pinned to scikit-learn $\geq 1.7.2, < 1.8$. It was trained earlier on raw URL text and copied into this backend. It was not retrained here.

The model scores only the raw URL string. Structured pre-scan features are counted and stored, but they are not model inputs. Training labels `good` and `bad` are shown as `legitimate` and `phishing`. The stored result is `evidence.ml_analysis`:

- `phishing_probability`
- `predicted_class`
- `model_type` (`LogisticRegression`) and `model_version` (`1.0`)
- `features_used`, up to 24 recognized URL tokens
- `model_available`
- a short summary

The dashboard shows this block as **ML analysis** and states that it is not the final verdict. The OpenAI instructions repeat that rule: a high phishing probability is one signal and must not become the verdict by itself.

A missing, corrupt, or failing model returns `model_available: false` and an error string. The browser, technical, reputation, and AI steps still run.

The saved model can assign a high phishing probability to an ordinary keyword-heavy URL. Read it beside the other evidence. Reputation checks can still produce **Malicious** when this classifier says legitimate.

Browser Evidence Collection

- Uses Playwright and Chromium in headless mode by default.
- Opens each URL in a fresh browser context.
- Follows navigation and records HTTP redirect URLs, locations, and status codes.
- Records the final resolved URL and final response status.
- Saves a full-page PNG screenshot per analysis.
- Saves downloaded HTML with configurable byte limits and truncation metadata.
- Applies configurable browser navigation timeouts and redirect limits.
- Blocks local, private, link-local, multicast, reserved, special-use, and unresolved network destinations before a browser starts and while it is loading page resources.

Technical Analyzers

Each analyzer is isolated in its own backend module and contributes standardized evidence items.

Analyzer	Evidence collected
Redirect analyzer	Redirect count, URL transitions, cross-host redirects, and related risk signals.
HTML analyzer	Page structure and potentially unusual or limited HTML content.

Analyzer	Evidence collected
Metadata analyzer	Titles, descriptions, canonical data, and relevant page metadata.
Forms analyzer	Forms, credential-oriented fields, methods, actions, and external form targets.
JavaScript analyzer	Inline scripts and suspicious JavaScript indicators. <code>location.replace()</code> is the redirect pattern. Assigning <code>location.href</code> is not treated as that pattern.
DNS analyzer	DNS resolution information for the destination hostname.
SSL analyzer	HTTPS certificate connectivity and certificate details.
WHOIS analyzer	Registered-domain information and age-related evidence where available.
Reputation analyzer	VirusTotal and Google Safe Browsing outcomes, isolated from technical evidence.

Reputation Services

- VirusTotal URL reputation integration using the v3 URL report endpoint.
- Google Safe Browsing v4 URL lookup integration.
- Independent provider execution so one failed provider does not discard the other provider's result.
- Configurable provider timeout handling.
- Explicit evidence for disabled API keys, malformed responses, HTTP failures, and timeouts.
- Reputation checks use the browser's final URL when one is available.

Google Safe Browsing v4 is intended for non-commercial use. A commercial deployment should use Google Web Risk or another appropriately licensed provider.

AI Verdict Generation

- Integrates the OpenAI Responses API through an async client.
- Sends a normalized, collected evidence bundle to the configured model.
- Uses strict JSON-schema structured output for predictable application data.
- Produces:
 - Verdict: `Safe`, `Suspicious`, or `Malicious`
 - Risk score: integer from 0 to 100
 - Explanation
 - Recommended user action
 - Confidence score
 - Weighted evidence rationale
- Transparently reports provider name, model, response ID, completion status, and whether a fallback was used.
- Uses a conservative local heuristic when `OPENAI_API_KEY` is missing, OpenAI is unavailable, or the response cannot be validated.
- Instructs the model that pre-scan keywords and `ml_analysis.phishing_probability` are evidence only and must not, by themselves, make a URL malicious.

- Maps the fallback from the strongest technical or reputation severity: info 8, low 25, medium 50, high 75, and critical 92. A reputation hit raises the fallback score to at least 92. A failed browser capture raises it to at least 65. The same bands apply: 0–30 Safe, 31–69 Suspicious, and 70–100 Malicious. The result reports `fallback_used: true`.

API and Persistence

- FastAPI application with CORS configured for the Vite development server.
- Async SQLAlchemy with SQLite and `aiosqlite`.
- Analysis jobs include the submitted and normalized URL, status, verdict, score, explanation, evidence JSON, error data, and timestamps.
- Screenshot retrieval verifies that the requested job owns the expected artifact before serving it.
- Development OpenAPI and ReDoc endpoints are available through FastAPI when the environment is not production.

Dashboard

- React, TypeScript, Vite, and Tailwind CSS frontend.
- URL input with an in-progress scanning state.
- Safe, Suspicious, and Malicious verdict badges and risk-score bar.
- Human-readable explanation, job metadata, and validation details.
- Initial URL risk from the pre-scan, labeled as not the final verdict.
- ML analysis from the local classifier, including probability, predicted class, model name, and the same not-the-final-verdict label. An unavailable model shows its error instead.
- Browser-capture details, redirect timeline, final URL, and screenshot preview.
- AI assessment details, including confidence, action, model, status, fallback state, and top evidence weights.
- Technical and reputation evidence sections.
- Retry controls for transport errors and persisted failed jobs.
- Responsive layout and a light/dark theme toggle.

Quality and Developer Experience

- Type hints and strict mypy configuration in the backend.
- Ruff linting for backend quality checks.
- Pytest coverage for validation, API lifecycle, browser error mapping, technical analyzers, reputation integrations, and OpenAI response handling.
- Pytest coverage for the URL pre-scan, the local classifier, and the rule that keyword-heavy URLs and a high phishing probability do not replace the final verdict.
- Vitest and Testing Library coverage for dashboard rendering, the pre-scan and ML blocks, successful scans, failure states, retries, screenshot behavior, and theme toggling.
- Environment-based configuration with `.env.example` templates.
- Standard application logging to stdout.
- Structured JSON logs with correlation IDs, method/path/status/duration fields, and no URL query strings.
- Per-client scan throttling, request-body size limits, and security response headers.
- Browser-level Playwright coverage for the compiled dashboard.

Architecture

ClickSafe follows clean-architecture dependency direction: framework and external-service code depends on application services; application services depend on domain contracts; domain models remain framework independent.

```

Browser UI (React + TypeScript + Tailwind)
  |
  v
FastAPI routes and schemas
  |
  v
Application services and URL validation
  |
  +--> Domain models and evidence contracts
  |
  +--> Analyzers
  |
  +--> Infrastructure adapters
        |- SQLite repository
        |- Playwright Chromium client
        |- Local phishing classifier
        |- OpenAI Responses API client
        |- VirusTotal client
        `-- Google Safe Browsing client

```

The pre-scan and classifier run inside the application service, before validation and before any adapter visits the URL.

Repository Layout

```

clickSafe/
|- backend/
|  |- src/clicksafe/
|  |  |- api/           FastAPI routers and dependency wiring
|  |  |- application/  URL validation, pre-scan, classifier, and
|  |  | orchestration
|  |  |- analyzers/    One phishing analyzer per file
|  |  |- core/         Settings and logging
|  |  |- domain/       Job, verdict, and evidence contracts
|  |  |- infrastructure/ Browser, database, AI, local model, and reputation
|  |  | adapters
|  |  `-- schemas/     Request and response models
|  `-- tests/          Backend unit and API tests
|- frontend/
|  `-- src/
|     |- components/   Scanner, shell, and result UI
|     |- lib/          Backend API client
|     |- types/        TypeScript analysis contracts
|     `-- __tests__/   Frontend tests

```

```
| - docs/
|   ` - architecture.md      Architecture documentation
| - README.md               Quick start and current endpoint overview
| - ROADMAP.md              Phase-by-phase implementation plan
| - PROGRESS.md             Beginner-friendly snapshot of what is finished
| ` - PROJECT_DETAILS.md     This complete project reference
```

The runtime model file is `backend/src/clicksafe/infrastructure/ml/models/model.joblib`. The older training project is kept in this repository as `previous_ml_project/`. It is ordinary source now. It is not a nested Git repository, and the current app does not run it.

API Contract

Base path: `/api/v1`

Method	Path	Purpose
GET	<code>/health</code>	Returns application health, name, version, and environment.
POST	<code>/analyze</code>	Validates, scans, analyzes, persists, and returns one analysis job.
GET	<code>/analyses?limit=20</code>	Returns recent analysis jobs. The limit range is 1 to 100.
GET	<code>/analyses/{analysis_id}</code>	Returns a saved job by ID.
GET	<code>/analyses/{analysis_id}/screenshot</code>	Returns the analysis PNG when its expected artifact exists.

Analyze Request

```
{
  "url": "https://example.com"
}
```

Analyze Response Shape

```
{
  "id": "uuid",
  "submitted_url": "https://example.com",
  "normalized_url": "https://example.com/",
  "status": "completed",
  "verdict": "Safe",
  "risk_score": 8,
  "explanation": "Human-readable risk explanation.",
  "evidence": {
    "pre_scan": {},
  }
}
```

```
    "ml_analysis": {},
    "validation": {},
    "browser": {},
    "technical_analysis": [],
    "reputation": [],
    "ai": {},
    "lifecycle": {},
    "pending_capabilities": []
  },
  "error_message": null,
  "created_at": "ISO-8601 timestamp",
  "updated_at": "ISO-8601 timestamp",
  "completed_at": "ISO-8601 timestamp"
}
```

The **evidence** object is intentionally extensible. Current top-level groups are:

- **pre_scan**: local URL-text score, level, features, keywords, brand matches, and warnings.
- **ml_analysis**: local classifier probability, class, model metadata, and availability.
- **validation**: normalized URL data and validation result.
- **browser**: capture status, final URL, redirects, artifact paths, HTML size, and truncation.
- **technical_analysis**: standardized analyzer evidence items.
- **reputation**: reputation evidence and per-provider results.
- **ai**: provider state, verdict, score, explanation, confidence, recommendation, and evidence weights.
- **lifecycle**: internal job progress metadata.

Configuration

Copy the root **.env.example** to **.env** for local configuration. Do not commit real API keys.

Variable	Purpose	Default
APP_NAME	API display name	ClickSafe API
APP_ENV	development, test, or production	development
APP_VERSION	Application version	0.1.0
LOG_LEVEL	Server log level	INFO
API_V1_PREFIX	API base path	/api/v1
DATABASE_URL	Async SQLite connection URL	sqlite+aiosqlite:///./clicksafe.db
SQL_ECHO	Emit SQL statements	false

Variable	Purpose	Default
OPENAI_API_KEY	Enables AI verdict generation	empty
OPENAI_MODEL	Responses API model	gpt-5.6
OPENAI_REASONING_EFFORT	Model reasoning setting	medium
OPENAI_TIMEOUT_SECONDS	AI request timeout	30
OPENAI_MAX_OUTPUT_TOKENS	Maximum AI output tokens	1200
VIRUSTOTAL_API_KEY	Enables VirusTotal lookup	empty
GOOGLE_SAFE_BROWSING_API_KEY	Enables Google Safe Browsing lookup	empty
REPUTATION_TIMEOUT_SECONDS	Timeout per reputation provider	10
PLAYWRIGHT_HEADLESS	Run Chromium without a visible window	true
BROWSER_TIMEOUT_MS	Browser navigation timeout	15000
MAX_REDIRECTS	Browser redirect limit	10
MAX_HTML_BYTES	Maximum persisted HTML artifact size	1000000
SCREENSHOT_DIR	PNG artifact directory	data/screenshots
HTML_DIR	HTML artifact directory	data/html
BLOCK_PRIVATE_NETWORKS	Blocks restricted	true

Variable	Purpose	Default
	browser destinations	
DNS_RESOLUTION_TIMEOUT_SECONDS	Destination DNS lookup timeout	5
MAX_REQUEST_BODY_BYTES	API request-body size cap	8192
ANALYSIS_RATE_LIMIT_ENABLED	Enables scan throttling	true
ANALYSIS_RATE_LIMIT_REQUESTS	Scans allowed per client window	5
ANALYSIS_RATE_LIMIT_WINDOW_SECONDS	Scan throttling window	60
CORS_ORIGINS	Comma-separated allowed frontend origins	local Vite URLs
VITE_API_BASE_URL	Frontend API origin	http://127.0.0.1:8000

Local Development

Backend

```
cd backend
python -m venv .venv
.\\.venv\\Scripts\\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -e ".[dev]"
python -m playwright install chromium
uvicorn clicksafe.main:app --reload
```

The backend normally listens on <http://127.0.0.1:8000>. Interactive API documentation is at <http://127.0.0.1:8000/docs> in development mode.

Frontend

```
cd frontend
npm install
```

```
npm run dev
npm run test:e2e
```

The Vite development server normally listens on <http://localhost:5173>.

Verification Commands

```
# From backend with the virtual environment active
pytest
ruff check src tests
mypy src

# From frontend
npm run test
npm run build
```

Technology Stack

Area	Technology
Backend API	Python 3.12+, FastAPI, Uvicorn
Validation and settings	Pydantic, pydantic-settings
Database	SQLite, SQLAlchemy async, aiosqlite
Browser automation	Playwright, Chromium
HTML processing	Beautiful Soup
DNS, WHOIS, and TLS	dnspython, python-whois, tldextract, cryptography
Local classifier	scikit-learn 1.7.x, joblib
Reputation	VirusTotal API v3, Google Safe Browsing v4
AI reasoning	OpenAI Python SDK and Responses API
Frontend	React 19, TypeScript, Vite, Tailwind CSS
Icons	lucide-react
Testing	pytest, pytest-asyncio, Vitest, Testing Library
Static checks	Ruff, mypy, TypeScript compiler

Completed Hardening

Phase 8: Production Hardening

Status: **complete**.

- Added per-client scan throttling and request-body size controls.

- Added SSRF protection for localhost, private, link-local, reserved, special-use, and unresolved destinations before browser navigation and during browser requests.
- Added JSON logs with correlation IDs and URL-query redaction by design.
- Added browser-level end-to-end dashboard coverage.
- Added [deployment guidance](#) and operational checks.

Present in the Repository, Not on the Live Scan

These modules exist and are not called by `AnalysisService.analyze`:

- `PaymentAnalyzer` in `backend/src/clicksafe/analyzers/payment.py`. The default analyzer list does not include it.
- The agent package in `backend/src/clicksafe/application/agent/`. It is not wired to the dashboard scan.

Security and Operational Notes

- Keep ClickSafe behind a reverse proxy and in a dedicated scanning environment. Application-level SSRF protection is active, but browser egress policy remains the final security boundary.
- A fresh Playwright browser context improves browser-session isolation but is not a complete network-security boundary. Production deployment should use a dedicated sandboxed worker with restricted egress and resource limits.
- Reputation services receive the URL being checked. Do not submit URLs containing secrets or sensitive query parameters without understanding that third-party disclosure.
- Store `.env`, SQLite databases, screenshots, and captured HTML outside version control.
- Screenshot and HTML artifacts can contain sensitive webpage content. Define retention and deletion policies before production use.
- The backend serves screenshots only after checking that the path matches the analysis ID's expected artifact path.
- Keep API keys server-side. The frontend should only know the backend base URL.

Project Completion Criteria

ClickSafe has completed its committed implementation roadmap. The URL pre-scan and local classifier are additional evidence inside that completed pipeline. Before a production launch, the team must still review provider licensing, secret management, artifact retention, network isolation, and operational ownership for its specific environment.

Related Documents

- [Quick start and endpoint overview](#)
- [Beginner-friendly progress snapshot](#)
- [Phase-by-phase roadmap](#)
- [Architecture notes](#)
- [Deployment guide](#)
- [Backend-specific commands](#)